Keywords: discrete diffusion • mixture of experts • fast text generation • open-weight LLM

DiffusionGemma Technical Report

Google DeepMind's Open 26B MoE Model Generates Text at 1000+ Tokens per Second via Block-Parallel Discrete Diffusion

By Adrien Ali Taïga et al. (DiffusionGemma Team, Google DeepMind)

arXiv:2608.00146 • August 2026

Autoregressive text generation has a fundamental throughput ceiling: it emits one token per forward pass, making latency proportional to sequence length. *DiffusionGemma* eliminates this bottleneck by iteratively denoising an entire 256-token canvas in parallel, achieving over 1000 tokens per second on a single NVIDIA H100—up to 4× faster than comparable AR models—while fitting within 18 GB VRAM when quantised and released under the Apache 2.0 licence.

AT A GLANCE

Problem: Autoregressive LLMs generate one token per forward pass, making throughput fundamentally linear in sequence length and bottlenecked by memory bandwidth.

Why it matters: A 4× throughput gain at matched quality halves the inference cost of frontier LLMs, directly enabling larger, cheaper deployments.

Method: Fine-tune the Gemma 4 26B MoE backbone with (1) SFT for bidirectional denoising and (2) RL + sampler distillation for joint quality and efficiency gains.

Result: 1000+ tok/s on H100, 700+ tok/s on RTX 5090; fits in 18 GB VRAM when quantised; open-weights under Apache 2.0.

Evidence: Speed benchmarks on NVIDIA H100 and RTX 5090; quality evaluated against equivalent-parameter AR models.

The Big Picture

Large language models generate text by sampling one token at a time in a strict left-to-right order. This autoregressive (AR) constraint is simple and ensures the model can condition each new token on all prior context, but it imposes a hard throughput ceiling: no matter how wide the model is, each forward pass produces exactly one token. For a sequence of N tokens, the model must run N serial passes, making decode time $O(N)$ in the number of forward passes regardless of hardware parallelism.

Diffusion-based text generation breaks this constraint by treating generation as iterative denoising over a complete sequence. DiffusionGemma represents the state of the art on this front: a 26B-parameter open-weights model that generates full blocks of tokens in parallel, with measured throughput exceeding 1000 tokens per second on a single H100—a 4× improvement over equivalent AR models.

Why Existing Approaches Fall Short

Earlier attempts at diffusion-based language models (e.g. MDLM, SEDD) trained from scratch on text corpora and did not match the quality of AR models of similar parameter count. The quality gap arose because diffusion LMs require learning bidirectional context modelling, a fundamentally harder pre-training problem than the unidirectional next-token prediction that gives AR models their strong priors.

Prior fast-generation methods for AR models (speculative decoding, token streaming) provide modest speedups (2–3×) but remain bounded by the sequential auto-regressive paradigm. DiffusionGemma bypasses this bound entirely.

Core Mechanism

Architecture. DiffusionGemma is based on Gemma 4, specifically the 26B-A4B Mixture-of-Experts variant: 26B total parameters with only 3.8B activated per token during inference. The sparse MoE routing means most expert parameters are dormant in any given forward pass, keeping latency low while retaining the model's large capacity.

Block-Autoregressive Multi-Canvas Sampling. At inference, DiffusionGemma begins with a canvas of 256 `[MASK]` tokens. Each denoising iteration:

1. Predicts the probability distribution over the full vocabulary for every masked position simultaneously;
2. Samples and commits the most confident subset of positions;
3. Repeats until the canvas is fully populated.

Once committed, positions are not revised. In practice far fewer than 256 iterations are required, as many positions are filled confidently in the first few steps.

Technical Formulation

Let $\mathbf{x} = (x_1, \ldots, x_L)$ be a sequence of L tokens. At each denoising step t the model receives a partially masked sequence $\tilde{\mathbf{x}}^{(t)}$ and predicts a distribution over unmasked completions:

$$p_\theta\left(\mathbf{x} \mid \tilde{\mathbf{x}}^{(t)}\right) = \prod_{i \in \mathcal{M}^{(t)}} p_\theta\left(x_i \mid \tilde{\mathbf{x}}^{(t)}\right),$$

where $\mathcal{M}^{(t)}$ is the set of masked positions at step t. The SFT objective is:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{\mathbf{x},t,\mathcal{M}}\left[\sum_{i\in\mathcal{M}} \log p_\theta(x_i \mid \tilde{\mathbf{x}}^{(t)})\right].$$

The RL stage optimises a reward $r(\mathbf{x})$ that combines generation quality (from a judge model) and inference efficiency (number of denoising steps to convergence):

$$\mathcal{L}_{\text{RL}}(\theta) = -\mathbb{E}_{\mathbf{x}\sim p_\theta}[r(\mathbf{x})] + \lambda\, D_{\text{KL}}(p_\theta \| p_{\text{ref}}),$$

where p_{ref} is the SFT checkpoint and λ controls KL regularisation.

Learning or Inference Procedure

Stage 1—SFT. Starting from the Gemma 4 AR checkpoint, the model is fine-tuned on text data using the masked denoising objective above. Less than 10% of the original pre-training token budget is required, as the AR checkpoint provides strong representations that transfer directly to the bidirectional denoising task.

Stage 2—RL + Sampler Distillation. A combined training step jointly (i) uses RL to improve response quality by rewarding high-judge-score outputs and (ii) distils a fast sampler that fills the canvas in fewer denoising steps without quality loss.

Inference. The model is applied with block-autoregressive sampling: a canvas of 256 masked tokens is initialised, then denoised iteratively. Generation can be quantised to 4-bit precision to fit within 18 GB VRAM.

What the Guarantee Says

The two-stage recipe inherits Gemma 4's general language understanding and instruction following, while the RL stage provides an explicit optimisation target linking generation quality to reward. The KL regularisation in the RL objective bounds the distance from the SFT checkpoint, preventing degenerate solutions that score high on the judge but collapse the output distribution. The sampler distillation guarantee is that the fast sampler matches the full sampler's output distribution within a KL tolerance tuned during training.

Experimental Findings

- **1000+ tokens/second** on a single NVIDIA H100 (batch size 1, int4 quantisation).
- **700+ tokens/second** on a consumer NVIDIA GeForce RTX 5090.
- **Up to** 4× **faster** than the Gemma 4 AR baseline on dedicated GPU hardware.
- Fits in **18 GB VRAM** when quantised to 4-bit, enabling local deployment on high-end consumer GPUs without multi-GPU setup.
- Quality competitive with the equivalent AR model on standard instruction-following and language-understanding benchmarks.

Ablations and Interpretation

Stage 1 only vs. Stage 1+2. The SFT-only checkpoint has higher variance in denoising step count; Stage 2 distillation produces a sampler that converges in fewer iterations, explaining the efficiency gains beyond what the base denoiser achieves.

Canvas size. The 256-token canvas size balances between parallelism (larger is better) and the difficulty of predicting many tokens at once with high confidence (larger is worse). Ablations showed 256 to be near-optimal for the 26B parameter scale.

MoE vs. dense. The 26B-A4B MoE architecture activates fewer parameters per token than a dense 4B model would but leverages a much larger total capacity. The sparse activation is essential for hitting the latency targets at this parameter scale.

Reference

DiffusionGemma Team (Adrien Ali Taïga et al.). **DiffusionGemma Technical Report**. arXiv:2608.00146, August 2026. https://arxiv.org/abs/2608.00146

Keywords: tabular prediction • in-context learning • dimensionality • LLM limitations

Why Large Language Models Fail at Tabular Prediction

A Systematic Study Identifies Input Dimensionality—Not Format, Noise, or Tokenisation—as the Decisive Factor in LLM Tabular Failure

By Marta Garnelo & Wojciech M. Czarnecki (Google DeepMind)

arXiv:2608.02412 • Submitted 3 August 2026

Large language models are outperformed by fifty-year-old baselines on tabular data. A new controlled study across 31 benchmark datasets falsifies four popular explanations for this failure and finds that dimensionality alone is decisive: LLM accuracy monotonically falls as feature count grows, while every classical method stays flat or improves, and in two dimensions the LLM's predictions match a local distance-based method with up to 91.6% grid agreement.

AT A GLANCE

Problem: LLMs routinely lose to classical baselines on tabular prediction, but the root cause has been unknown.

Why it matters: Tabular data dominates enterprise ML; understanding why LLMs fail guides where not to deploy them and points to what a fix would require.

Method: Evaluate a frontier LLM in pure in-context inference on 31 datasets; sweep input dimensionality via random linear projections; test five candidate hypotheses.

Result: Dimensionality is the sole decisive factor; all four other hypotheses are falsified by controlled experiments.

Evidence: 31 datasets, 9 comparison methods; 91.6% prediction grid agreement with local distance-based methods in 2-D; failure mode unmatched by any classical degradation.

The Big Picture

Tabular data—rows of numeric and categorical features with a prediction target—is the most common format in real-world machine learning. Despite LLMs' remarkable across-the-board progress, they have failed to close the gap with classical methods on tabular benchmarks. A frontier LLM prompted with a CSV of training examples and asked to predict the label for a test row is consistently beaten by random forests, gradient boosting, and even simple k-nearest-neighbour classifiers.

This failure has spawned a dedicated field of tabular foundation models, yet the underlying cause has been debated without controlled evidence. Garnelo and Czarnecki run the most systematic study to date, falsifying four popular hypotheses and identifying the sole culprit: input dimensionality.

Why Existing Approaches Fall Short

The field has proposed several ad-hoc explanations:

- **Noisy / non-separable data:** LLMs may be sensitive to label noise or overlapping class boundaries more common in tabular data than text.
- **CSV format:** Serialising rows as comma-separated values may obscure the column structure that classical methods exploit.
- **Numeric tokenisation:** Numbers like `3.14` are split into multiple tokens (`3`, `.`, `14`), possibly disrupting numeric reasoning.
- **Query batch size:** Classifying many test points per prompt may introduce interference between predictions.

None of these has been rigorously tested as a *cause* of the failure. All four are falsified below.

Core Mechanism

The study places the LLM in its purest in-context regime: a single generation pass over a prompt containing the full training set serialised as CSV plus one test point. No tools, fine-tuning, chain-of-thought, or agentic scaffolding. This isolates the LLM's native prediction mechanism.

To control dimensionality, random linear projections are applied to each of the 31 datasets, producing versions with 2, 4, 8, 16, 32, and 64 features. This lets the study vary dimensionality while keeping the underlying dataset structure the same.

The LLM is compared to eight classical methods including k-NN, random forests, gradient-boosted trees, and linear classifiers—nine methods total.

Technical Formulation

Let $\mathbf{X} \in \mathbb{R}^{n \times d}$ be the training set with labels $\mathbf{y} \in \{1, \ldots, C\}^n$, and let $\mathbf{x}^* \in \mathbb{R}^d$ be a test point. The LLM predicts:

$$\hat{y}^* = \arg\max_c \, p_{\text{LLM}}(c \mid \text{prompt}(\mathbf{X}, \mathbf{y}, \mathbf{x}^*)).$$

The dimensionality sweep uses a random matrix $\mathbf{P} \in \mathbb{R}^{d \times k}$ with $k < d$ to project features: $\tilde{\mathbf{x}} = \mathbf{x}\mathbf{P}$. Accuracy is

measured as a function of k.

For the behavioural comparison, the *prediction grid agreement* between two classifiers f_1 and f_2 is:

$$\text{Agreement}(f_1, f_2) = \mathbb{E}_{\mathbf{x}^*}[\mathbf{1}[f_1(\mathbf{x}^*) = f_2(\mathbf{x}^*)]]\,,$$

evaluated on a fine grid over the feature space.

Learning or Inference Procedure

The experimental protocol has three phases:

1. **Hypothesis testing:** For each of the five hypotheses, design a controlled variant of the prompt or data (e.g., corrupt labels for (a), use alternative serialisation for (b)) and test whether it changes the gap to classical methods. No change = hypothesis falsified.
2. **Dimensionality sweep:** Apply random projections to sweep $d \in \{2, 4, 8, 16, 32, 64\}$ and measure accuracy vs. d for all nine methods.
3. **Behavioural comparison:** In 2-D (where the LLM still performs competitively), compute prediction grid agreement between the LLM and all eight classical methods to identify which method's decision boundary it mimics.

What the Guarantee Says

Falsification results are definitive at the experimental level: each variant produces no statistically significant change in the LLM–classical gap when hypotheses (a)–(d) are tested. The dimensionality effect is monotone across all 31 datasets and all 9 comparison methods, making it robust to dataset-specific confounds.

Experimental Findings

Dimensionality is decisive. The LLM is the only method among nine whose accuracy decreases monotonically with d. Every classical baseline stays flat or improves as d grows (more features contain more signal for classical methods). The crossover point where classical methods overtake the LLM is typically around $d = 4$–8.

In 2-D, the LLM mimics a distance-based method. Grid agreement between the LLM and the nearest-neighbour classifier reaches **91.6%** in 2-D. No other classical method achieves this level of agreement with the LLM's predictions at any dimensionality.

High-dimensional failure mode is novel. At $d > 8$, the LLM's decision boundary no longer matches any classical method, including noise-corrupted or degraded variants. The failure is mechanistically distinct from all classical failure modes.

Ablations and Interpretation

Hypothesis (a)—noise: Adding label noise to all methods equally reduces accuracy for all, but the LLM–classical gap is unchanged. Falsified.

Hypothesis (b)—CSV format: Alternative serialisations (JSON, Markdown tables, space-separated) produce no statistically significant change in LLM accuracy. Falsified.

Hypothesis (c)—tokenisation: Replacing all numeric values with symbolic tokens (`A`, `B`, ...) does not restore LLM performance. Falsified.

Hypothesis (d)—batch size: Classifying 1, 5, or 20 test points per query produces no consistent change in accuracy. Falsified.

The prediction mechanism at high d is an open question. The LLM's unique sensitivity to dimensionality and the breakdown of its resemblance to any known classical method at $d > 8$ leave its high-dimensional prediction mechanism as a fundamental open problem for mechanistic interpretability research.

Reference

Marta Garnelo, Wojciech M. Czarnecki. **Why Large Language Models Fail at Tabular Prediction**. arXiv:2608.02412, August 2026. https://arxiv.org/abs/2608.02412

Keywords: diffusion transformer • momentum orthogonalization • optimizer • image generation

CMuon: Accelerating and Stabilizing Diffusion Transformer Training via Chunked Momentum Orthogonalization

Partitioning Fused Weight Matrices Before Orthogonalization Delivers a 2× Speedup Over AdamW and FID 1.18 on ImageNet 256

By Chuyan Chen, Peng Sun & Kun Yuan

arXiv:2608.02502 • ECCV 2026

The Muon optimizer's momentum orthogonalization accelerates standard Transformer training, but applying it directly to Diffusion Transformers stalls at late stages. CMuon traces the cause to fused weight tensors coupling independent subspaces and fixes it with a single structural change—partitioning matrices into chunks before orthogonalization. A 675M-parameter DiT trained with CMuon achieves FID 1.18 on ImageNet 256×256 in 200 epochs, more than 2× faster than AdamW.

AT A GLANCE

Problem: Muon achieves faster convergence than AdamW on standard Transformers but stalls in the late training phase when applied to Diffusion Transformers (DiTs).

Why it matters: DiT training is the dominant compute cost for frontier image and video generation; a 2× optimizer speedup directly halves training cost.

Method: Identify fused weight tensors (AdaLN, QKV) as the source of subspace coupling; chunk them prior to Nesterov momentum orthogonalization.

Result: FID 1.18 on ImageNet 256×256 in 200 epochs (675M parameters), $>$ 2× training speedup over AdamW.

Evidence: Controlled ablations against Muon and AdamW; FID curves over full 200-epoch training run; accepted ECCV 2026.

The Big Picture

Diffusion Transformers (DiTs) power the best image and video generation models of 2026, from FLUX to Sora-class video synthesis. Training them is extremely expensive: a state-of-the-art image generation DiT requires thousands of GPU-hours, and video models demand an order of magnitude more. Any optimizer improvement that accelerates convergence without degrading final quality translates directly to lower training cost.

Muon—the Momentum Orthogonalization optimizer—has attracted attention because it achieves significantly faster convergence than AdamW on vanilla Transformer language models. CMuon makes Muon work on DiTs by identifying and fixing a structural mismatch between Muon's assumptions and DiT's weight layout.

Why Existing Approaches Fall Short

AdamW is the de-facto DiT optimizer. It works reliably but converges slowly, requiring many epochs to reach competitive FID scores.

Vanilla Muon applies Nesterov momentum and then orthogonalizes the resulting gradient update matrix via Newton–Schulz iterations before applying the weight update. This exploits the geometry of the weight matrix space to find descent directions that make maximal use of each parameter. On standard Transformer blocks (Q, K, V, and MLP projections each in their own tensors), it works well.

But DiTs use **Adaptive Layer Norm (AdaLN)**: a conditioning mechanism that generates per-channel scale and shift parameters from the diffusion timestep embedding, modulating the activations of each Transformer block. For GPU efficiency, the linear layers implementing AdaLN fuse the weights for scale and shift into a single large matrix, and similarly the Q, K, V projections are sometimes stored fused. Applying Muon to these fused tensors treats two functionally independent linear maps as a single entity, causing the orthogonalization to mix their update directions. This implicit coupling distorts the gradient geometry and causes convergence to plateau.

Core Mechanism

CMuon introduces a minimal pre-processing step before the Muon update:

1. **Identify** all fused weight tensors in the DiT (AdaLN scale+shift matrices, fused QKV projections, etc.).
2. **Chunk** each fused matrix along its output channel dimension according to the functional boundary (e.g., split AdaLN into its scale matrix and its shift matrix separately).
3. **Orthogonalize** each chunk independently via the Newton–Schulz approximation to the matrix orthogonal factor.
4. **Reassemble** the chunked updates and apply them to the original fused parameter tensor.

No architectural changes or additional hyperparameters are required. The same chunking boundaries used during the forward pass for functional interpretation are reused

here during the optimizer step.

Technical Formulation

Let $\mathbf{W} \in \mathbb{R}^{2d\times d}$ be a fused AdaLN weight matrix combining a scale projection $\mathbf{W}_s \in \mathbb{R}^{d\times d}$ and a shift projection $\mathbf{W}_b \in \mathbb{R}^{d\times d}$.

The Muon update for $\mathbf{W}$ is:

$$\Delta\mathbf{W} = \mathrm{NS}(\mathbf{M}), \quad \mathbf{M} = \mu\mathbf{M}_{\text{prev}} + \mathbf{G},$$

where $\mathbf{G}$ is the gradient, μ is momentum, and $\mathrm{NS}(\cdot)$ denotes the Newton–Schulz orthogonalization:

$$\mathrm{NS}(\mathbf{A}) \approx \frac{3}{2}\mathbf{A} - \frac{1}{2}\mathbf{A}\mathbf{A}^\top\mathbf{A} \quad \text{(one step)}.$$

The CMuon update orthogonalizes each chunk separately:

$$\Delta\mathbf{W}_s = \mathrm{NS}(\mathbf{M}_s), \quad \Delta\mathbf{W}_b = \mathrm{NS}(\mathbf{M}_b),$$

where $\mathbf{M}_s, \mathbf{M}_b$ are the momentum matrices for each chunk (maintained separately). The final update is $\Delta\mathbf{W} = [\Delta\mathbf{W}_s; \Delta\mathbf{W}_b]$.

By orthogonalizing each functional subspace independently, each chunk's update direction is computed in the geometry appropriate to its own weight space rather than the mixed geometry of the fused tensor.

Learning or Inference Procedure

CMuon is a drop-in optimizer replacement. Training proceeds as:

1. Forward pass and diffusion denoising loss computation (identical to standard DiT training).
2. Backward pass yielding gradients for all parameters.
3. For each fused weight tensor, split gradients and momentum along the chunk boundary, orthogonalize each chunk, reassemble update.
4. For non-fused parameters (e.g., biases, embedding tables), use AdamW as usual (Muon is not well-suited to 1-D parameter vectors).
5. Apply updates with a learning rate schedule identical to AdamW baselines for fair comparison.

Inference is unaffected: CMuon is a training-time optimizer; the deployed model is identical in architecture to a standard DiT.

What the Guarantee Says

Muon's orthogonalization is motivated by the Riemannian gradient on the manifold of matrices with bounded spectral norm. Each update step is guaranteed to be a descent direction in this geometry. The CMuon insight is that applying this guarantee to a fused matrix provides a weaker guarantee than applying it to each component separately, because the fused matrix does not lie on a single manifold coherent with either component's optimization landscape. Chunk-level orthogonalization restores the per-component descent guarantee.

Experimental Findings

Evaluated on DiT-XL/2 (675M parameters) training on ImageNet 256×256 with class-conditional generation:

- CMuon achieves **FID 1.18** in 200 epochs—competitive with the best published DiT-XL/2 results.
- Training is $> 2\times$ **faster** than AdamW measured by FID at equal epoch count.
- Vanilla Muon on DiTs plateaus around epoch 120–140 and never reaches FID 1.3; CMuon continues descending through 200 epochs.
- No guidance (classifier-free or otherwise) is used during evaluation, making this an apples-to-apples comparison.

Ablations and Interpretation

Without chunking (vanilla Muon): Convergence stalls at FID ≈ 1.4 around epoch 130. The late-stage plateau is eliminated by chunking.

Chunk granularity: Chunking at the AdaLN scale+shift boundary is sufficient. Further splitting within Q, K, V provides marginal additional gain on this architecture.

Non-fused layers: Applying Muon (without chunking) to non-fused DiT layers (attention projections stored separately) does not cause convergence issues, confirming that fused tensors are the specific failure mode.

AdamW vs. CMuon FID curves: At epoch 100 AdamW achieves FID $\approx$1.8, while CMuon achieves FID $\approx$1.3, confirming the $2\times$ epoch-efficiency claim.

Reference

Chuyan Chen, Peng Sun, Kun Yuan. **CMuon: Accelerating and Stabilizing Diffusion Transformer Training via Chunked Momentum Orthogonalization**. arXiv:2608.02502, ECCV 2026. https://arxiv.org/abs/2608.02502

Keywords: multimodal reasoning • auxiliary memory • chain-of-thought • training-free inference

TRAM: Enhancing Multimodal Reasoning with Trajectory-Derived Auxiliary Memory

A Training-Free Inference Wrapper Recovers Reasoning-Derived Information Lost in Long Multimodal Chains Without Any Fine-Tuning

By Kang Liu, Zijing Wang, Yongkang Liu, Mengjie Zhao, Xiaocui Yang, Shi Feng, Yifei Zhang & Daling Wang

arXiv:2608.01922 • Submitted 3 August 2026

As multimodal reasoning chains grow long, intermediate conclusions lose influence over later steps. *TRAM* fixes this without retraining: a training-free inference wrapper extracts a compact trajectory memory from each intermediate step and injects it into decoding, closing the accuracy gap caused by long-chain information loss in Multimodal Large Reasoning Models (MLRMs).

AT A GLANCE

Problem: In long reasoning chains, intermediate conclusions and visual inferences established early in the trajectory lose influence over later decoding steps, causing accuracy to degrade.

Why it matters: Hard visual reasoning benchmarks require chains of many steps; state-of-the-art MLRMs degrade precisely on these long-chain cases where the capability would matter most.

Method: Training-free auxiliary memory extracted from the reasoning trajectory so far and injected as additional context at each decoding step.

Result: Improved accuracy on long-chain multimodal reasoning benchmarks without fine-tuning or model weight modification.

Evidence: Attribution analysis showing correctness correlates with trajectory retention, not visual attribution alone; benchmark improvements across multiple MLRMs.

The Big Picture

Multimodal Large Reasoning Models (MLRMs) combine a vision encoder with a large language model to tackle tasks that require reading an image, constructing multi-step inferences, and arriving at a final answer. Representative models include LLaVA-o1, InternVL-Reasoner, and similar 2026 state-of-the-art systems.

These models work well on short reasoning tasks but degrade on long chains—precisely the cases where step-by-step reasoning is most valuable. TRAM provides a training-free remedy, applicable as a plug-in wrapper around any deployed MLRM.

Why Existing Approaches Fall Short

Retraining with longer context windows is expensive and risks catastrophic forgetting of other capabilities. Models with 32K+ context windows still degrade on long reasoning tasks because the issue is not available context length—the intermediate conclusions *are* in the context—but effective attention to them.

Fine-tuning on curated long-chain examples is data-hungry, hard to generalise, and requires modifying model weights. This prevents deployment on proprietary models accessed via API.

Prompt engineering (e.g., adding explicit "remember that..." prompts) is fragile and task-specific.

TRAM avoids all of these by operating as an inference-time wrapper with no access to model weights.

Core Mechanism

TRAM rests on an attribution insight: correctness in MLRMs is not well-predicted by visual token attribution (how much the model "looks at" the image), but by whether the trajectory up to each step retains and integrates earlier reasoning-derived information. This means the bottleneck is not the visual encoder—it is the propagation of intermediate conclusions through the reasoning chain.

TRAM addresses this by maintaining an explicit **trajectory memory** $\mathcal{M}$ that accumulates reasoning-derived information as generation proceeds:

1. At each completed reasoning step s_t, a lightweight extractor parses the step and identifies key propositions (visual relations, numeric facts, logical constraints, sub-conclusions).
2. These are added to $\mathcal{M}$ indexed by type and step index.
3. When generating the next step s_{t+1}, the most relevant entries from $\mathcal{M}$ are retrieved and prepended to the decoding context as an "Auxiliary Memory" block.

The extractor and retriever are lightweight and rule-based; no neural modules beyond the MLRM itself are required.

Technical Formulation

Let $\mathcal{H}_t = (v, s_1, s_2, \ldots, s_t)$ be the full trajectory at step t, where v is the visual input and each s_i is a reasoning step. The MLRM without TRAM generates:

$$s_{t+1} \sim p_\theta(s_{t+1} \mid \mathcal{H}_t).$$

TRAM augments this with trajectory memory $\mathcal{M}_t$, a structured summary of $\mathcal{H}_t$:

$$s_{t+1} \sim p_\theta(s_{t+1} \mid \mathcal{H}_t, \mathcal{M}_t)\,,$$

where $\mathcal{M}_t$ is constructed by:

$$\mathcal{M}_t = \bigoplus_{i=1}^{t} \text{Extract}(s_i),$$

with Extract parsing the step text and $\bigoplus$ denoting structured accumulation into the memory store.

The attribution score used to motivate the design is:

$$A_{\text{traj}}(s_j, s_{t+1}) = \sum_k \alpha_{k,j}^{(t)},$$

where $\alpha_{k,j}^{(t)}$ is the attention weight from position k in step s_{t+1} to token j in a prior step. The empirical finding is that A_{traj} for early steps decreases as t grows, explaining the long-chain accuracy degradation.

Learning or Inference Procedure

TRAM operates as a post-processing wrapper around any MLRM's generation loop:

1. Load the MLRM (weights unmodified).
2. Initialise $\mathcal{M}_0 = \emptyset$.
3. For each reasoning step t:
   (a) Retrieve top-K entries from $\mathcal{M}_{t-1}$ relevant to the current partial step (keyword + type matching).
   (b) Prepend retrieved entries as an "Auxiliary Memory" block to the MLRM's context.
   (c) Sample s_t from the augmented context.
   (d) Run Extract(s_t) and add results to $\mathcal{M}_t$.
4. Return the final answer from the last reasoning step.

No gradient computation or parameter update occurs at any step.

What the Guarantee Says

The method provides no formal convergence guarantee, but the attribution analysis provides empirical evidence for the causal chain: (i) early trajectory attention decays with chain length; (ii) TRAM injects a compressed representation of the decayed information; (iii) accuracy on long-chain benchmarks improves.

The compression step (Extract) is necessarily lossy, so TRAM provides benefit primarily when the extracted propositions are those the model would have failed to recover from the diluted attention—a condition satisfied empirically for the benchmark tasks studied.

Experimental Findings

Evaluated across multiple MLRMs on long-chain multimodal reasoning benchmarks:

- Consistent accuracy improvement on long-chain reasoning tasks (those requiring > 6 reasoning steps) across all tested MLRMs.
- Minimal or no improvement on short-chain tasks, consistent with the theory that attention dilution is not a bottleneck when chains are short.
- Improvement is larger for tasks requiring explicit recall of visual relations or numeric facts established early in the chain.
- Wall-clock overhead: the extractor and memory retrieval add negligible latency relative to the MLRM's own generation time.

Ablations and Interpretation

Memory type: All three memory entry types (visual relations, numeric facts, logical constraints) contribute; removing any one reduces gains by 15–30%.

Retrieval top-K: Performance peaks at $K = 3$–5; beyond $K = 8$ the auxiliary block becomes too long and begins to hurt attention focus.

Extractor quality: Replacing the rule-based extractor with an LLM-based extractor improves results slightly but increases latency. The rule-based version provides most of the gain at negligible cost.

Baseline—full context: Simply prepending the full $\mathcal{H}_t$ history twice does not help; the structured compression step is essential.

Reference

Kang Liu, Zijing Wang, Yongkang Liu, Mengjie Zhao, Xiaocui Yang, Shi Feng, Yifei Zhang, Daling Wang. **TRAM: Enhancing Multimodal Reasoning with Trajectory-Derived Auxiliary Memory**. arXiv:2608.01922, August 2026. https://arxiv.org/abs/2608.01922

Keywords: rectified flow • optimal transport • flow matching • statistical learning theory

Computational and Statistical Guarantees of the c-Rectified Flow

A Cost-Aware Flow Matching Variant Provably Minimises User-Specified Transport Costs While Preserving Marginals, With Sample Complexity Bounds

By Leda Wang, Zhehao Xu, Qiang Liu & Harrison H. Zhou (UT Austin; Yale University)

arXiv:2608.02487 • August 2026

Flow matching trains fast generative models but offers no guarantee that the learned transport minimises any user-specified cost. The *c-Rectified Flow* restricts learned vector fields to gradient-of-convex-conjugate form and proves that iterative application monotonically reduces c-transport cost while automatically preserving marginal distributions—with sample complexity bounds characterising how many data points are needed.

AT A GLANCE

Problem: Standard rectified flow / flow matching learns a valid generative model but provides no guarantee of minimising any user-specified transport cost.

Why it matters: Many scientific applications (molecule generation, image editing, optimal control) have explicit cost functions; a theoretically grounded cost-aware flow model is essential for safe deployment.

Method: Restrict vector fields to $\nabla\bar{c}(\nabla f_t(\cdot))$ form; iterative application provably reduces c-transport cost.

Result: Computational and statistical guarantees for cost-aware flow learning; conditions for recovery of the true optimal transport solution.

Evidence: Formal proofs of convergence and sample complexity; counterexamples delineating when strong assumptions are necessary.

The Big Picture

Rectified flow and its generalisations (flow matching, stochastic interpolants) have become the standard training framework for generative models used in image synthesis, video generation, and scientific modelling. The core idea is simple: train a neural ODE velocity field $v_t(x)$ to push samples from a source distribution π_0 (e.g., Gaussian noise) towards a target distribution π_1 (e.g., data), with training implemented as a simple regression objective.

This works remarkably well in practice. But for applications where the *how* of transport matters—molecule generation (minimise conformational energy), image editing (preserve perceptual structure), optimal control (minimise actuator effort)—standard flow matching provides no guarantee that the learned transport is cost-efficient. It may find a valid coupling $\pi_0 \to \pi_1$ that is wasteful or even physically unrealisable.

c-Rectified Flow closes this gap.

Why Existing Approaches Fall Short

Standard rectified flow minimises the squared-L_2 distance between a straight-line trajectory and the learned trajectory, which corresponds to the L_2 transport cost. It provides no guarantees for other cost functions and even for L_2 it recovers the OT solution only under strong regularity conditions.

Iterative rectification (reflow) applies rectified flow recursively on coupled samples to straighten trajectories, reducing L_2 cost in practice but providing no monotone convergence guarantee for general costs c.

Minibatch OT pre-computes approximate couplings before training, but the approximation error accumulates and there is no learning-theoretic analysis of the resulting flow.

Core Mechanism

For a convex cost function $c : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$, let $\bar{c}$ denote its convex conjugate (Legendre–Fenchel transform):

$$\bar{c}(u) = \sup_x \langle u, x\rangle - c(x).$$

c-Rectified Flow restricts the learned velocity field to the structured form:

$$v_t(x) = \nabla\bar{c}(\nabla f_t(x))\,,$$

where $f_t : \mathbb{R}^d \to \mathbb{R}$ is an unconstrained time-dependent scalar potential. The key observation is that any vector field of this form is automatically the subdifferential of a convex function (since $\bar{c}$ is convex and f_t is smooth), which is precisely the structure of the optimal transport map for cost c by Brenier's generalised theorem.

The neural network learns f_t (unconstrained), and v_t is computed from it via automatic differentiation. The constraint is enforced implicitly by the parameterisation, not by projection or regularisation.

Technical Formulation

Training objective. Given source samples $\{x_0^{(i)}\}_{i=1}^n \sim \pi_0$ and target samples $\{x_1^{(j)}\}_{j=1}^n \sim \pi_1$, paired by a coupling γ_t at interpolation time $t \in [0,1]$, the c-Rectified

Flow objective is:

$$\mathcal{L}_{c\text{-RF}}(\theta) = \mathbb{E}_{(x_0,x_1)\sim\gamma}\left[\|v_\theta(x_t,t) - \dot{x}_t\|^2\right], \quad x_t = (1-t)x_0 + tx_1,$$

where $v_\theta = \nabla\bar{c}(\nabla f_\theta)$ is enforced by parameterisation.

Computational guarantee. Let $\mathcal{T}_c(\gamma)$ denote the c-transport cost of coupling γ. Theorem 1 in the paper establishes:

$$\mathcal{T}_c(\gamma^{(k+1)}) \le \mathcal{T}_c(\gamma^{(k)})$$

for all k, where $\gamma^{(k)}$ is the coupling induced by the k-th iterate of c-rectification. Marginals are preserved exactly at each iteration.

Statistical guarantee. Theorem 2 provides: with n i.i.d. samples from $\pi_0 \times \pi_1$, the estimated c-transport cost satisfies:

$$\mathcal{T}_c(\hat{\gamma}_n) - \mathcal{T}_c(\gamma^*) \le C_s \cdot n^{-2/(d+4)} \cdot \log n$$

with high probability, where C_s depends on the smoothness class of the optimal transport map and the Lipschitz constant of $\bar{c}$.

Learning or Inference Procedure

Iterative c-rectification:

1. Initialise coupling $\gamma^{(0)} = \pi_0 \otimes \pi_1$ (independent).
2. For $k = 0, 1, 2, \ldots$:
   (a) Pair samples from π_0 and π_1 by simulating the current flow $v^{(k)}$ to obtain coupled pairs $(x_0, x_1^{(k)})$.
   (b) Train $v^{(k+1)}$ by minimising $\mathcal{L}_{c\text{-RF}}$ on these coupled pairs.

Inference: Simulate the learned ODE $\dot{x}_t = v_\theta^{(K)}(x_t, t)$ from $x_0 \sim \pi_0$ to x_1 using a numerical ODE solver (e.g., Euler with T steps).

What the Guarantee Says

The computational guarantee ensures that repeated c-rectification is a monotone interior algorithm: each iterate reduces the cost without leaving the feasible set of couplings with the correct marginals. This is analogous to coordinate descent on a convex objective but here the geometry is the space of probability couplings.

The statistical guarantee implies that at dimension d and smoothness s, the estimation error decays as $n^{-s/(2s+d)}$ (classical non-parametric rate), showing that c-Rectified Flow is statistically efficient.

However, recovery of the true OT solution requires additional assumptions: connected support of π_0, π_1 and smoothness of the OT map T^*. The paper provides counterexamples showing these conditions are not vacuous.

Experimental Findings

- On synthetic 2-D optimal transport problems with non-L_2 costs (e.g., $c(x,y) = \|x-y\|_1$, earth-mover cost), c-Rectified Flow achieves provably lower transport cost than standard flow matching after the same number of training iterations.
- Monotone cost decrease is confirmed empirically for $K = 1, 2, 3$ iterations of c-rectification across five experimental settings.
- Sample complexity curves match the theoretical $n^{-2/(d+4)}$ rate on synthetic data with known ground-truth OT maps.
- On a molecular conformer generation task with steric clash cost, c-Rectified Flow produces conformers with lower energy than standard flow matching baselines.

Ablations and Interpretation

Effect of $\bar{c}$ parameterisation: Using a learned neural $\bar{c}$ rather than the closed-form convex conjugate produces slightly better results but is harder to optimise and risks losing the theoretical guarantees if the learned function is not truly convex.

Iteration count K: Cost decreases most sharply in the first iteration ($k = 0 \to 1$) and yields diminishing returns beyond $K = 3$. This matches the theoretical prediction that later iterates are already close to the OT solution.

Disconnect: theory vs. practice: In settings without smooth OT maps (e.g., multimodal targets with disconnected support), c-Rectified Flow converges to a local cost minimum rather than the global OT solution, consistent with the counterexamples in the theoretical analysis.

Reference

Leda Wang, Zhehao Xu, Qiang Liu, Harrison H. Zhou. **Computational and Statistical Guarantees of the c-Rectified Flow**. arXiv:2608.02487, August 2026. https://arxiv.org/abs/2608.02487